

Ofren Collect — User Stories

Project: Ofren Collect — Recurring billing + automatic reconciliation on Monnify **Author:** Ofren Technologies **Context:** APIConf Lagos 2026 × Monnify Developer Challenge **Companion to:** REQUIREMENTS.md

How to read this

Stories follow the form "As a <role>, I want <capability>, so that <benefit>." Each has acceptance criteria and a priority (MoSCoW), and links back to the functional requirement IDs it realises. Stories are grouped into epics.

Personas

- **Bola — the business owner (tenant Owner).** Runs a small SaaS / gym / tutoring service with 30–80 recurring customers. Not technical. Currently tracks payments in a spreadsheet and chases people manually. Signs in to her own isolated account (tenant). Ofren Collect's primary user.
 - **Tunde — another business owner (a different tenant).** Exists in these stories only to make one thing testable: Bola must *never* see Tunde's data, and vice-versa. Tenant isolation is a first-class requirement, not an afterthought.
 - **Chidi — the customer.** Pays Bola every month. Wants a simple, reliable way to pay and to not be wrongly chased.
 - **The system** — acts on Bola's behalf automatically (webhooks, reconciliation, flagging, auditing). Some stories are written from the system's perspective because the automation *is* the product.
 - **The judge / evaluator.** Needs to clone, run, and understand the project quickly. A real user for this build.
-

Epic 0 — Sign in and stay isolated

Goal: Each business has its own secure, private account; no business can see another's data.

US-0.1 — Register my business · Must · (FR-0.1)

As a new business owner, I want to sign up and get my own account, so that I can start managing my collections privately.

Acceptance criteria - Registering creates a tenant and an owner user in one step. - I can immediately log in with the credentials I set.

US-0.2 — Log in securely · Must · (FR-0.2, FR-0.3)

As a business owner, I want to log in with email and password and receive a session token, so that only I can access my data.

Acceptance criteria - Correct credentials return a signed token carrying my tenant and role. - Passwords are stored only as salted hashes. - Wrong credentials are rejected without revealing which part was wrong.

US-0.3 — Everything I do stays within my business · Must · (FR-0.4, FR-0.5, FR-0.6)

As a business owner, I want every action I take to apply only to my own data, so that I can trust the system with my business.

Acceptance criteria - All business endpoints require a valid token; the tenant is taken from the token, not from anything I send. - I can read and change only my own tenant's plans, customers, subscriptions, and invoices. - There is no request parameter that lets me reach another tenant's data.

US-0.4 — I never see another business's data · Must · (FR-0.6, FR-0.7)

As a business owner (Bola), I want to be certain I can't see Tunde's customers, payments, or live updates, so that the platform is safe to use.

Acceptance criteria - Queries return only the caller's tenant's rows (enforced at the data layer, not per-query). - Real-time notifications reach only the tenant they belong to.

US-0.5 — Payments find the right business automatically · Must · (FR-0.8)

As the system, I want to determine which tenant an incoming Monnify payment belongs to from the reserved account it landed in, so that webhooks (which carry no login) still respect tenant boundaries.

Acceptance criteria - A verified webhook resolves to the owning subscription and therefore tenant via the reserved account. - The resulting records are stamped with that tenant.

US-0.6 — Sensitive actions are restricted · Should · (FR-0.9)

As a business owner, I want sensitive views like the audit log limited to the Owner role, so that not every user can see everything.

Acceptance criteria - Owner-only endpoints reject non-owner tokens.

Epic 1 — Set up what I charge

Goal: Bola can define her recurring charges.

US-1.1 — Create a plan · Must · (FR-1.1, FR-1.3)

As a business owner, I want to create a plan with a name, amount, and billing interval, so that I can define what I charge and how often.

Acceptance criteria - I can enter a plan name, an amount in naira, and choose an interval (e.g. monthly). - The plan is rejected if the amount is zero/negative or the name is blank. - The saved plan appears in my list of plans.

US-1.2 — See my plans · Must · (FR-1.2)

As a business owner, I want to see all the plans I've created, so that I can pick the right one when enrolling a customer.

Acceptance criteria - All active plans are listed with name, amount, and interval.

Epic 2 — Put customers on a plan

Goal: Bola can enrol a customer and give them a way to pay.

US-2.1 — Register a customer · Must · (FR-2.1)

As a business owner, I want to add a customer with their name and email, so that I can bill them.

Acceptance criteria - I can save a customer with name and email. - The customer is available to enrol on a plan.

US-2.2 — Enrol a customer and get them a payment account · Must · (FR-2.2, FR-2.3, FR-2.4, FR-2.6)

As a business owner, I want to enrol a customer onto a plan and automatically get a dedicated account number for them to pay into, so that I don't have to manually track who paid.

Acceptance criteria - Enrolling a customer creates a subscription. - A dedicated Monnify reserved account is provisioned for that subscription. - The account number and bank are stored and shown to me so I can share them with the customer. - Each subscription has its own unique account reference.

US-2.3 — First invoice is created automatically · Must · (FR-2.5)

As a business owner, I want the first invoice to be created when I enrol a customer, so that there's something for their payment to settle against.

Acceptance criteria - On enrolment, an invoice is created with the plan amount and a due date based on the interval.

US-2.4 — Cancel a subscription · Should · (FR-2.7)

As a business owner, I want to cancel a subscription, so that a customer who has left stops being invoiced and chased.

Acceptance criteria - A cancelled subscription generates no new invoices. - It is visually distinct from active subscriptions.

Epic 3 — Take payments without manual matching

Goal: When a customer pays, the system knows — safely and automatically.

US-3.1 — Customer pays into their own account · Must · (FR-2.6)

As a customer, I want a fixed account number to pay into each period, so that paying is simple and familiar.

Acceptance criteria - The reserved account number stays the same across periods for the same subscription.

US-3.2 — System detects an inflow · Must · (FR-3.1, FR-3.2)

As the system, I want to receive a notification the moment money lands in a reserved account, so that reconciliation can happen without anyone checking manually.

Acceptance criteria - A webhook endpoint receives Monnify transaction notifications. - It acknowledges promptly with HTTP 200 so Monnify does not retry unnecessarily.

US-3.3 — System trusts only verified payments · Must · (FR-3.3, FR-3.4, NFR-1.4, NFR-1.5)

As the system, I want to authenticate the webhook and independently verify the transaction with Monnify before acting, so that no payment is recorded on the basis of an unverified or spoofed message.

Acceptance criteria - An incoming webhook fails closed if its signature/hash is invalid. - The transaction is confirmed via Monnify's verification endpoint before reconciliation.

US-3.4 — No double-counting · Must · (FR-3.5, FR-3.6, NFR-2.1)

As the system, I want to record each transaction once and ignore repeats of the same reference, so that a re-delivered webhook never inflates what a customer has paid.

Acceptance criteria - Each transaction reference is stored once. - Re-processing the same reference has no additional effect.

Epic 4 — Know exactly who has paid

Goal: The reconciliation that replaces Bola's spreadsheet.

US-4.1 — Match a payment to the right customer · Must · (FR-4.1, FR-4.2)

As a business owner, I want each payment automatically attributed to the right customer and invoice, so that I never manually match inflows again.

Acceptance criteria - An inflow is resolved to its subscription via the reserved account it landed in. - The amount is applied to that subscription's current open invoice.

US-4.2 — Full payments settle cleanly · Must · (FR-4.3)

As a business owner, I want an invoice marked paid when the full amount arrives, so that I can see at a glance who is settled.

Acceptance criteria - Invoice status becomes **Paid** when paid amount equals amount due.

US-4.3 — Short payments are caught · Must · (FR-4.4)

As a business owner, I want to know when a customer has underpaid and by how much, so that I can follow up for the balance instead of assuming they're settled.

Acceptance criteria - Invoice status becomes **Underpaid** when paid amount is below amount due. - The outstanding balance is recorded.

US-4.4 — Overpayments become credit · Should · (FR-4.5)

As a business owner, I want an overpayment recorded as credit, so that a customer who pays too much isn't shortchanged next period.

Acceptance criteria - Invoice status becomes **Overpaid** when paid amount exceeds amount due. - The surplus is recorded as credit.

US-4.5 — Billing rolls forward · Should · (FR-4.6)

As a business owner, I want the next invoice created automatically once one is fully paid, so that recurring billing continues without my involvement.

Acceptance criteria - On full payment, the subscription advances to the next period and a new invoice is created with the correct due date.

US-4.6 — Stray payments aren't lost · Should · (FR-4.7)

As a business owner, I want payments that don't match any subscription surfaced for review, so that money is never silently dropped.

Acceptance criteria - An inflow matching no subscription is recorded as **Unmatched** and shown for review, not discarded.

US-4.7 — Overdue customers are flagged · Should · (FR-4.8)

As a business owner, I want overdue subscriptions flagged automatically, so that I know exactly who to chase without checking dates myself.

Acceptance criteria - A subscription whose invoice passes its due date unpaid is flagged **Overdue**.

Epic 5 — See the whole picture at a glance

Goal: The one screen that replaces the spreadsheet.

US-5.1 — See all subscriptions and their status · Must · (FR-5.1, FR-5.4)

As a business owner, I want one dashboard showing every subscription with its status, updating live as payments arrive, so that I understand my collections without digging or refreshing.

Acceptance criteria - The dashboard lists customer, plan, reserved account, next due date, and status — for my tenant only. - When a payment is reconciled, the affected row updates in real time without a manual refresh. - I can create plans and enrol customers from the dashboard.

US-5.2 — Read status at a glance · Should · (FR-5.2)

As a business owner, I want statuses colour-coded, so that I can spot problems (overdue, underpaid) instantly.

Acceptance criteria - Distinct visual treatment for paid, underpaid, and overdue states.

US-5.3 — See the headline numbers · Should · (FR-5.3)

As a business owner, I want summary figures — collected this period, count overdue, count unmatched — so that I get the health of my collections in one look.

Acceptance criteria - Summary strip shows total collected this period, overdue count, and unmatched count.

Epic 6 — Trust, visibility, and resilience

Goal: The system is safe to operate — every action is traceable, dependencies fail gracefully, and abuse is limited.

US-6.1 — Every action is recorded · Must · (FR-8.1, FR-8.2)

As a business owner, I want a record of every action taken in my account, so that I have accountability and can investigate anything unexpected.

Acceptance criteria - Each API call is logged with correlation id, user, tenant, method, path, status, and time. - Request/response bodies are recorded with secrets, tokens, and full account numbers redacted.

US-6.2 — I can review my audit trail · Should · (FR-8.3, FR-8.4)

As a business owner (Owner), I want to view my own account's activity log, so that I can see what happened and when.

Acceptance criteria - I can query my tenant's audit entries; I never see another tenant's. - A correlation id links an audit entry to the matching application logs.

US-6.3 — A Monnify hiccup doesn't break me · Should · (NFR-2.5)

As the system, I want to retry transient Monnify failures and stop calling Monnify when it's clearly down, so that a temporary outage doesn't fail operations or cascade.

Acceptance criteria - Transient failures are retried with backoff. - After repeated failures the circuit breaker opens (fails fast), then probes for recovery.

US-6.4 — Abuse and floods are limited · Should · (NFR-1.9)

As the system, I want to rate-limit requests, so that no user or IP can overwhelm the API or brute-force logins.

Acceptance criteria - Authenticated routes are limited per tenant/user; anonymous auth and webhook routes are limited per IP. - Over-limit requests receive 429 with a retry hint.

US-6.5 — No acknowledged payment is ever lost · Should · (NFR-2.6)

As the system, I want a verified webhook stored durably in the same transaction as the payment record, so that a crash mid-processing cannot drop a payment I already acknowledged.

Acceptance criteria - The notification and payment record commit together. - An interrupted notification is picked up and completed on restart.

Epic 7 — Settle and ask (stretch)

Goal: Nice-to-haves that demonstrate extra depth. Build only if the core and security layers are solid.

US-7.1 — Pay collected funds out · Could · (FR-6.1, FR-6.2)

As a business owner, I want to move collected funds to my bank account, so that money doesn't sit in reserved accounts.

Acceptance criteria - A settlement transfer to a merchant account can be triggered via Monnify. - The transfer is recorded with its reference and status.

US-7.2 — Ask a plain question about my money · Could · (FR-7.1, FR-7.3, FR-7.4, FR-7.5)

As a business owner, I want to ask "how much did I collect this week?" in plain language, so that I get answers without learning the interface.

Acceptance criteria - The assistant answers from my tenant's real data — it never invents figures. - It is read-only: it cannot create, change, cancel, or move money. - It respects my tenant's isolation, is audit-logged, and is rate-limited like any other endpoint. - It sits behind a feature flag and a provider-agnostic interface, so it can be turned off or swapped by configuration. - If it can't answer reliably, it is excluded from the submission rather than shipped half-working.

Epic 8 — Let an evaluator run it in minutes

Goal: The judge is a user too. These stories protect the submission score directly.

US-8.1 — Clone and run from the README alone · Must · (NFR-3.1, NFR-4.1, NFR-6.1)

As a judge, I want to clone the repo and run it locally following only the README, so that I can evaluate it without guesswork.

Acceptance criteria - Setup requires only documented steps. - The datastore runs via SQL Server Express — LocalDB or the documented free container — with schema and seed created automatically on first run.

US-8.2 — See a working app immediately · Must · (NFR-3.2)

As a judge, I want demo data present on first run, so that the app demonstrates value without me having to create everything first.

Acceptance criteria - A seeded tenant, plan, customer, and subscription exist on first launch, so I can log in and see a populated dashboard.

US-8.3 — Trust the repo is safe · Must · (NFR-1.1, NFR-1.2, NFR-6.2)

As a judge, I want no exposed secrets or broken links, so that the submission meets the challenge's baseline quality bar.

Acceptance criteria - No API keys/secrets in the repo or its history. - Only a placeholder `.env.example` is present. - No broken links in the documentation.

US-8.4 — Understand the story fast · Must · (NFR-3.3)

As a judge, I want a clear problem statement and architecture overview, so that I grasp what was built and why within a couple of minutes.

Acceptance criteria - The README states the problem, the solution, and how it works.

Sprint focus (3-day reality)

The scope now spans core reconciliation, security, and operational concerns, so build in **tiers** — each tier is only started when the one above it works. If time runs short, cut from the bottom.

Tier 1 — core reconciliation spine (the demo narrative, valid on its own):

US-1.1 → US-2.2 → US-2.3 → US-3.2 → US-3.3 → US-4.1 → US-4.2 → US-4.3 → US-5.1 → US-8.1 → US-8.3

Tier 2 — security (makes it real and multi-tenant):

US-0.1 → US-0.2 → US-0.3 → US-0.4 (then US-0.5 so webhooks respect tenancy)

Tier 3 — operational (visibility & resilience):

US-6.1 (audit) → US-6.4 (rate limiting) → US-6.3 (retry/breaker) → US-6.5 (durable webhooks)

Tier 4 — stretch, cut first:

US-7.1 (settlement) → US-7.2 (AI assistant)

Everything marked **Should** adds polish and score; everything **Could** is genuinely optional and should be cut before quality on any higher tier slips. The cardinal rule: a working Tier 1 + Tier 2 beats a broken attempt at all four.